

Drydock — 10-Minute Launch Script

Format: product-launch pitch, conversational, first person. **Target length:** ~1,450 spoken words ≈ 10:00 at 145 wpm. **Pairing:** each block maps to one slide in [deck.html](#). Advance on the cue. **Delivery notes:** lead with the answer, pause on the bold lines, let the diagrams breathe.

Slide 1 — Title · 0:00–0:30

This is Drydock. It builds working software from specifications — and, just as important, it keeps that software honest as it grows. For two years we've watched AI write code faster than anyone imagined. The problem was never speed. The problem is that you can't *reproduce* the result. Drydock fixes that with a real process.

Slide 2 — The gap · 0:30–1:30

Here's the gap in specification-driven design as it exists today. It gave us specifications — and quietly threw away twenty-five years of software best practices for actually shipping them. To a financial engineer that gap is obvious: a number you can't reproduce isn't a result, and **without a process you can't reproduce a build either**. So you point an agent at the spec, it writes a thousand lines, and three weeks later it's drifted from the code, the context is full of the wrong files, and yesterday's decisions are gone. **The bottleneck was never the model. It's the missing process.**

Slide 3 — The insight · 1:30–2:15

Drydock is built on one idea: **reproducible builds need a process — and that process already exists**. It's Agile. The Agile Manifesto is twenty-five years old this year; it's how developers ship at every major company. Drydock is Agile applied to AI delivery. This is not about sub-agents or prompt tricks — it's about the *behavior* of the system. The specification stays the single source of truth; the process is what makes the build repeatable.

Slide 4 — Introducing Drydock · 2:15–3:10

Drydock is a governed, Blueprint-driven delivery system — an installable Python CLI that runs an agile process called **SAIL**: Set up, Analyze, Implement, Loop. You import your notes or LLM-drafted specifications. Your agile team — the LLM — analyzes them and converts them into specification files, divided into features and stories. It plans a dependency graph, then builds working software, one context-optimized step at a time. From there the process keeps your results in sync with edits and change tickets, generates documentation that conforms to the spec, and measures build quality and drift. Let me walk you through it.

Slide 5 — You are the Commander · 3:10–3:50

Drydock changes your role. You're no longer the developer or the designer — **you are the Commander**. The Commander decides; in Agile terms, the Commander is the product owner. The LLM is your **Agile Best Practices Team**, and the QuarterDeck is how you talk to them clearly and precisely. That's the key move, and it's about the *behavior* of the system, not sub-agents or prompt engineering. A Compass holds your intent, a Ship's Log records every decision, and the QuarterDeck is where you command. You're not prompting a chatbot. You're directing a team.

Slide 6 — S: Set up · 3:50–4:15

Phase one, Set up — laying the keel. Three commands: `pipx install`, `drydock config`, `drydock init`. You point Drydock at a workspace and a build directory, pick your provider — Claude or Codex — and create a target. **Drydock runs on your existing subscription CLI.** No new API keys, no per-token billing. All you need is a subscription and an idea.

Slide 7 — A: Analyze · 4:15–5:15

Phase two, Analyze — charting the course. You import your raw material: markdown specs, source code, Spec Kit projects, loose notes. Then `drydock analyze` decomposes all of it — using agile practices — into stories, acceptance milestones, blockers, and open questions. And here's the governance: **if the team finds a blocker, it stops and asks.** It writes the questions down; you answer them in the QuarterDeck and re-run. The output is a *tentative* set of Features, Stories, and Spikes for you to review — nothing is committed until you say so.

Slide 8 — The QuarterDeck · 5:15–6:05

This is the core of the whole thing — the QuarterDeck. It's your **Agile web interface to the process**: an optimized communication path between you, the Commander, and the LLM team. It renders everything the team produced — the story hierarchy, the blockers, the questionnaires — and you approve, answer, or redirect. Your feedback is persisted and carried into every run. There's even a `drydock rigging compact` step that produces minimal context for each file, with separate Builder and User views. **Instead of guessing, you have an optimized path that twenty-five years of agile practice already worked out. The Commander controls — and the goal is working software you can iterate.**

Slide 9 — The Manifest · 6:05–7:00

When you're satisfied, `drydock plan` turns the analysis into Blueprints — typed specification files — and a Manifest. The Manifest is the part I'm most proud of. **It's a dependency graph of your entire build.** During planning, the LLM estimates Story Points — the real token cost — for every story, against that graph. Drydock then stacks exactly the right files into each build prompt: your intent, the relevant spec slice, the task — and nothing else. That's how you get reproducible builds. Context is engineered, not hoped for.

Slide 10 — I: Implement · 7:00–7:50

Phase three, Implement — sailing the frontier. `drydock build` walks the dependency graph and builds the runnable frontier, one context-optimized step at a time. Every step produces reviewable evidence you can verify against its acceptance criteria. And `drydock build score` measures delivery health across **seven dimensions** — spec completeness, test coverage, drift, and more — so you always know how far the code has wandered from the spec.

Slide 11 — Rigging · 7:50–8:25

Enterprise teams need their own standards baked in. That's **Rigging** — your branding, your stack rules, your conventions, injected into every build. And it's smart about context: a feature's builder gets the full specification; a feature's consumer gets only a compacted how-to-use version. They don't need to know how it works, only how to call it. That keeps context lean across a large codebase.

Slide 12 — L: Loop · 8:25–9:00

Phase four, Loop — the refit. Software is never done. `drydock refit` lets you change the application while keeping the Blueprint and the code aligned, two ways. One: you edit the specification files directly, and because Drydock tracks the git commit behind every built file, it rebuilds only what changed. Two: you raise change tickets against existing specs, built dynamically with minimized context. Either way the specification stays the source of truth, and drift stays minimal.

Slide 13 — Why it's different · 9:00–9:35

So why Drydock. Vibe coding gives you no spec and no process — speed, then drift. Spec-driven design gives you a specification, but still no process to reproduce the build. **Drydock's differentiator is a real process, and that process is Agile** — the Commander decides, the team develops, every decision is logged. The goal is simple and it's the same goal agile has always had: working software you can reliably iterate, with an easy path to rebuild it anytime.

Slide 14 — Get started · 9:35–10:00

Drydock installs in one line — `pipx install drydock-sdd`. Point it at a workspace, init your first target, and you're charting a course. It's specification-driven design that actually holds the line. The spec is open, the methodology is open, and I'm looking for Commanders to take it for a sail. Thanks for watching.

Recording checklist

- Open `deck.html` in a browser, press **F** for fullscreen, screen-record at 1080p+.
- Practice once for timing; each slide has a target end-time above. If you run long, trim Slides 11 and 13 first.
- Record audio separately if you can — narrate to the timings, then drop the slide advances onto the audio in your editor.
- Leave a 1-second beat after each bold line; those are the lines viewers will clip.